Chapter 7: GeoSpatial Data

Nature Inspired Optimisation for Delivery Problems (2022)

Lecture Notes

Chapter Overview

This chapter introduces the data types and structures that underpin modern digital mapping and geographic information systems (GIS). Understanding these foundations is essential before we can optimise delivery routes, because every optimisation algorithm ultimately relies on a faithful computer representation of the real-world road network.

Chapter Contents

Key Takeaway

Map data may be stored as raster images or as vector data. A rendering algorithm may produce raster images from vector data. Vector data is particularly useful because it can carry rich metadata — opening hours, speed limits, turn restrictions — which feed directly into graph-based routing models.

7.1 A Brief History of Mapping I

The desire to create accurate maps of our environment is one of the oldest human endeavours. Archaeological evidence suggests that humans have been producing maps of their surroundings since pre-historic times, and the tradition has continued without interruption ever since.

Early Cartographic Milestones

- The Babylonian *Imago Mundi* tablet (c. 600 BCE) is one of the earliest surviving world maps, showing Babylon at the centre surrounded by a circular ocean.
- The Greek geographer Ptolemy (c. 150 CE) produced his *Geographia*, which introduced the concept of a coordinate grid (latitude and longitude) to precisely locate places.
- Following the invention of mechanical printing in the 15th century, maps became commodities that could be mass-produced, traded, and updated — a dramatic increase in the speed of knowledge transfer.

7.1 A Brief History of Mapping II

The Ordnance Survey and scientific cartography. As surveying techniques matured, nations invested heavily in systematic, large-scale mapping programmes. In the United Kingdom, the *Ordnance Survey* (OS) was founded in 1791. The OS still produces authoritative topographic maps today, and its datasets are widely used for navigation and infrastructure planning.

Two Core Principles from Ordnance Survey

- **Surveying accuracy**: OS maps are measured from a fixed network of triangulation points, so every location has a precisely known coordinate.
- **Two-pronged offering**: OS provides both (a) *map data* (raw numbers describing locations and features) and (b) *mapping software* that renders those numbers into images humans can read.

7.1 A Brief History of Mapping III

John Snow's 1854 cholera map. One of the most celebrated early uses of spatial data analysis is physician John Snow's investigation of a cholera outbreak in the Soho district of London. By plotting the home addresses of cholera victims on a street map, Snow discovered that cases clustered tightly around a single water pump on Broad Street. His insight — that the *spatial pattern* of a disease reveals its cause — was revolutionary. He argued that cholera was a water-borne illness at a time when the prevailing medical view held that it spread through "bad air" (miasma).

Worked Example: Snow's Spatial Reasoning

- Snow drew a map with each cholera death marked as a small bar stacked at the victim's address.
- He then drew concentric circles around the Broad Street pump.
- The vast majority of deaths fell within a short walking distance of that pump.
- Removal of the pump handle ended the outbreak — a classic demonstration that a map, combined with careful reasoning, can drive a life-saving decision.

7.1 A Brief History of Mapping IV

Booth's Poverty Maps (1889). Charles Booth produced a series of maps of London in which every street was colour-coded according to the socio-economic status of its inhabitants. This project — arguably the first large-scale social survey in history — showed that geospatial data could illuminate inequalities, not just physical terrain.

7.1 A Brief History of Mapping V

Fig. 7.2 Mercator's 1569 map of the world [Public Domain PD-US] Author: Gerardus Mercator (1569) https://commons.wikimedia.org/wiki/File:Mercator_1569.png

maps to visualise data allowing conclusions to be drawn that would not have been obvious by other means.

The development of printing techniques allowed for the widespread production and distribution of maps and atlases during the twentieth century. The use of maps by individuals for navigation and leisure use became widespread in many countries. The development of computing hardware and software in the second half of the twentieth century lead to the development of the *Geographical Information System* (GIS) Goodchild (2018); Tomlinson and Inventory (1967).

A Graphical Information System uses computers to process, store and visualise geographical data. As data storage and graphical capabilities increased GIS were developed to allow users to browse maps on screen and have data overlaid onto the maps. Other common GIS functions include the ability to search data on a geospatial basis, e.g. find me the closest restaurant to a specific location. Other useful functions of GIS include finding routes between locations in order to generate driving directions. Until the advent of the world wide web, GIS remained expensive and specialised. Early users of GIS included utility organisations to keep track of

An extract from Booth's 1889 Descriptive Map of London Poverty. Different colours represent different wealth categories,

7.2 Raster Graphics I

Before we can talk about storing or manipulating map data, we need to understand the two fundamental ways in which *any* graphical information can be represented in a computer: as a **raster image** or as **vector data**.

Definition: Raster Image

A raster image is a rectangular grid of individual picture elements called **pixels** (short for "picture elements"). Each pixel stores a colour value — for example, as three numbers representing the intensities of Red, Green, and Blue light (an RGB triplet). The full image is nothing more than a very large 2D array of such numbers.

Resolution and the zoom problem. The visual quality of a raster image is determined by its **resolution** — the number of pixels per unit area. A high-resolution image has more pixels and therefore more spatial detail. However, raster images suffer from a fundamental limitation: if you zoom in beyond the native resolution, the individual pixels become visible as blocky squares, and the image looks degraded. This is because the image literally contains no more information at finer scales; the data has been discarded at capture time.

7.2 Raster Graphics III

Common Raster File Formats

- **PNG** (Portable Network Graphics): lossless compression, supports transparency, excellent for maps with sharp edges and text.
- **JPEG** (Joint Photographic Experts Group): lossy compression, small file sizes, best for photographs but introduces artefacts at sharp boundaries.
- **TIFF** (Tagged Image File Format): lossless, supports metadata and multiple layers; widely used in professional GIS.
- **GeoTIFF**: a TIFF that embeds geographic coordinate information, so each pixel can be mapped to a real-world location.

7.2 Raster Graphics IV

Map tiles. Web mapping services such as Google Maps and OpenStreetMap serve their raster data as a pyramid of pre-rendered image tiles. At each zoom level, the world is divided into a grid of square tiles (typically 256×256 pixels each). When you pan or zoom in the browser, the client downloads only the tiles that are currently visible. This tile-pyramid approach makes it feasible to deliver planetary-scale raster data to billions of users on demand.

7.2 Vector Graphics I

Rather than storing a dense grid of pixel colours, a **vector** representation stores the *mathematical description* of geometric shapes: points, lines, curves, and polygons, each defined by a set of coordinates.

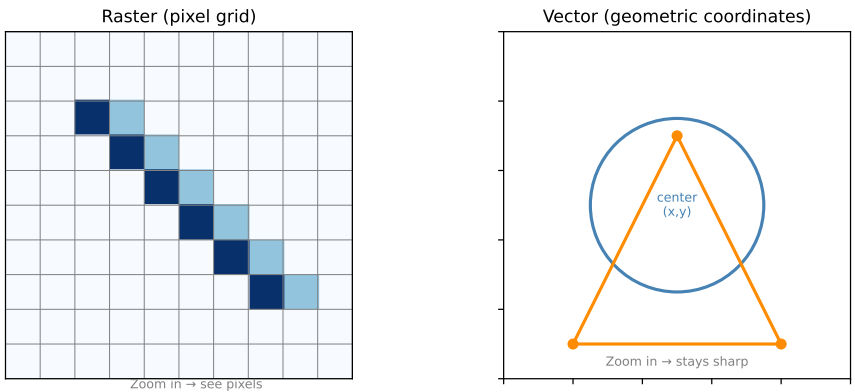
Definition: Vector Graphics

In a vector image, every feature is described by

- the **type** of shape (point, line, polygon, arc),
- the **coordinates** of its control points or vertices,
- optional **attributes** such as stroke colour, fill colour, and line width.

When the image is displayed, a *rendering engine* converts these geometric descriptions into the pixel grid that the screen can show.

Infinite zoom, finite metadata. Because a vector representation stores the geometry analytically, zooming in does not degrade quality — the rendering engine simply re-computes the rasterised output at the new scale. More importantly for GIS applications, each feature in a vector file can carry arbitrary **metadata**: a road segment might store its name, road classification (motorway, A-road, footpath), speed limit, and the opening times of shops along it.



Left: A raster image zoomed in reveals individual pixels. Right: A vector image is defined by coordinates and remains sharp at any zoom level.

Common Vector File Formats

- **SVG** (Scalable Vector Graphics): an XML-based format designed for the web. A circle is literally written as `<circle cx="100" cy="100" r="50"/>`. Because it is plain text, SVG files are easy to create, edit, and embed in web pages.
- **GeoJSON**: a JSON-based standard for encoding geographic features (points, lines, polygons) together with their non-spatial attributes. Widely used in web GIS and APIs.
- **Shapefile**: the longstanding industry standard from Esri, used in desktop GIS software such as ArcGIS. A Shapefile is actually a collection of at least three files (`.shp`, `.dbf`, `.shx`) stored together.
- **OSM (OpenStreetMap) XML**: an open format that encodes the full OSM data model — nodes, ways, and relations — in XML. This is the format used when you download an area from OpenStreetMap for offline use.

7.2 Vector Graphics IV

Common Vector Data Formats

Format	Extension	Human-readable?	Use case	Typical size
SVG	.svg	Yes (XML)	Web/print maps	Small–Medium
GeoJSON	.geojson	Yes (JSON)	Web GIS, APIs	Medium
Shapefile	.shp+	No (binary)	Desktop GIS	Large
OSM	.osm	Yes (XML)	OpenStreetMap	Very large
KML	.kml	Yes (XML)	Google Earth	Medium

GeoJSON: a concrete example. The following snippet shows how a road segment might be encoded as a GeoJSON `LineString` feature. Notice how geometry (the coordinate pairs) and metadata (road name, speed limit) live side-by-side in the same document.

Python: Reading a GeoJSON Vector File

Listing 1: GeoJSON structure and reading it with Python

```
import json

# A minimal GeoJSON file representing a road segment
geojson_text = """
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "LineString",
        "coordinates": [
          [-3.19, 55.95],
          [-2.90, 55.87],
          [-0.13, 51.51]
        ]
      },
      "properties": {
        "name": "A1 Road",
        "maxspeed": 70,
        "highway": "trunk"
      }
    }
  ]
}
```

Python: Reading OSM Data with OSMnx

OpenStreetMap data can be downloaded and converted into a Python graph structure using the `osmnx` library. The library handles all the complexities of the OSM data model and returns a `networkx` graph object that is immediately usable for routing.

Listing 2: Downloading and inspecting an OSM road network

```
import osmnx as ox

# Download the road network for a named place
# 'drive' means only driveable streets are included
G = ox.graph_from_place('Edinburgh, UK', network_type='drive')

# Basic statistics
print(f"Nodes (junctions): {len(G.nodes)}")
print(f"Edges (road segments): {len(G.edges)}")

# Each edge has attributes loaded from the OSM vector data
# For example, inspect the first edge:
u, v, data = list(G.edges(data=True))[0]
print(f"Edge {u} -> {v}")
print(f"  Street name : {data.get('name', 'unnamed')}")
print(f"  Length (m)  : {data.get('length', '?')}")
print(f"  Max speed   : {data.get('maxspeed', 'not set')}")
print(f"  Highway type: {data.get('highway', '?')}")

# Save the graph as a shapefile (vector format) for use in GIS tools
ox.save_graph_shapefile(G, filepath='edinburgh_graph')
```

7.3 Coordinate Systems: WGS84 I

A **coordinate system** is a mathematical framework that assigns a unique pair (or triple) of numbers to every point on Earth. Without a well-defined coordinate system, two people trying to specify the same location might use incompatible numbers.

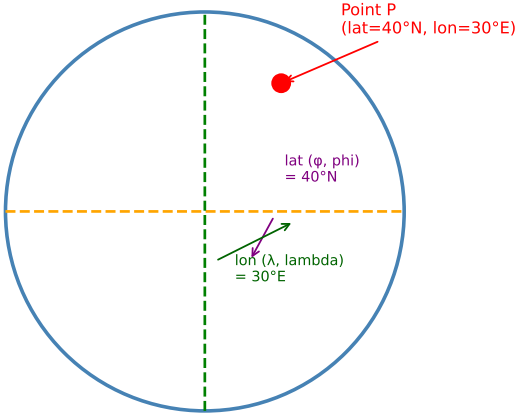
Definition: WGS84 (World Geodetic System 1984)

WGS84 is the global coordinate standard used by GPS (Global Positioning System) and by most modern mapping applications. It models the Earth as an oblate spheroid — a sphere that is very slightly flattened at the poles — and defines

- **Latitude** ϕ (phi): the angle north or south of the equator (the imaginary line midway between the poles). Ranges from -90° (South Pole) to $+90^\circ$ (North Pole). The equator is $\phi = 0^\circ$.
- **Longitude** λ (lambda): the angle east or west of the *prime meridian*, an arbitrary reference line passing through Greenwich, London. Ranges from -180° to $+180^\circ$.
- **Altitude** h: the height above the WGS84 reference ellipsoid, in metres.

7.3 Coordinate Systems: WGS84 II

WGS84: Latitude and Longitude



7.3 Haversine Distance Formula I

When we know the latitude and longitude of two points on the Earth's surface, we want to compute the straight-line *great-circle distance* between them — the shortest path along the Earth's curved surface, as opposed to a path through the interior of the planet.

Why not just use Pythagoras? If the two points are very close together (a few kilometres), using the Euclidean (flat-earth) formula gives a reasonable approximation. But for longer distances, the curvature of the Earth introduces significant errors. The Haversine formula accounts for this curvature exactly.

7.3 Haversine Distance Formula II

The Haversine Formula

Given two points with coordinates (ϕ_1, λ_1) and (ϕ_2, λ_2) (all angles in *radians*), define:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\,\cos(\phi_2)\,\sin^2\left(\frac{\Delta\lambda}{2}\right)$$

where $\Delta\phi = \phi_2 - \phi_1$ (delta-phi, the difference in latitude) and $\Delta\lambda = \lambda_2 - \lambda_1$ (delta-lambda, the difference in longitude). Then:

$$c = 2\,\mathrm{atan2}\left(\sqrt{a},\,\sqrt{1-a}\right)$$

$$d = R \times c$$

where $R \approx 6{,}371\,\mathrm{km}$ is the mean radius of the Earth, c (the central angle, in radians) is the angular separation of the two points as seen from the Earth's centre, and d is the great-circle distance in kilometres.

7.3 Haversine Distance Formula III

Notation Guide

- ϕ (phi): latitude in radians. To convert from degrees: $\phi = \text{degrees} \times \pi/180$.
- λ (lambda): longitude in radians.
- $\Delta\phi$ (delta-phi): the difference in latitude between the two points, in radians.
- $\Delta\lambda$ (delta-lambda): the difference in longitude, in radians.
- sin (sine): a trigonometric function. $\sin(\theta)$ (theta) ranges between -1 and $+1$.
- cos (cosine): another trigonometric function. $\cos(\theta)$ also ranges between -1 and $+1$.
- $\operatorname{atan2}(y, x)$: the "two-argument arctangent", which gives the angle whose tangent is y/x, taking the correct quadrant into account.
- R: the mean radius of the Earth, approximately 6,371 km.
- a: an intermediate value. It always lies in the range $[0, 1]$. When $a = 0$, the two points coincide; when $a = 1$, they are exactly antipodal (on opposite sides of the Earth).
- c: the central angle in radians. Multiplying by R converts it to a distance.
- d: the final great-circle distance in km.

How to Read This Formula

Python: Computing Haversine Distance

Listing 3: Haversine distance function in Python

```python
import math

def haversine(lat1: float, lon1: float,
              lat2: float, lon2: float) -> float:
    """
    Compute the great-circle distance between two points
    on the Earth's surface using the Haversine formula.

    Parameters
    ----------
    lat1, lon1 : float
        Latitude and longitude of point 1 in DEGREES.
    lat2, lon2 : float
        Latitude and longitude of point 2 in DEGREES.

    Returns
    -------
    float
        Great-circle distance in kilometres.
    """
    R = 6371.0  # Earth's mean radius in km

    # Convert degrees to radians
    phi1    = math.radians(lat1)
    phi2    = math.radians(lat2)
```

7.4 Graph Theory: Foundations I

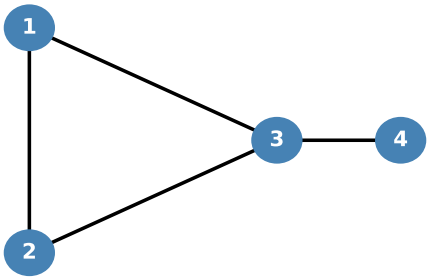
Now that we know how to store geographic locations as coordinates, we need a data structure that represents the *connectivity* of a road network. A road network consists of junctions (where streets meet) connected by road segments (the streets themselves). The mathematical object that captures exactly this structure — a set of items with connections between them — is called a **graph**.

Definition: Graph $G = (V, E)$

A graph G is formally defined as an ordered pair (V, E) where:

- V (capital V) is the **set of vertices** (also called *nodes*). Each vertex represents an entity — a road junction, a city, a person in a social network, etc.
- E (capital E) is the **set of edges** (also called *links* or *arcs*). Each edge represents a relationship or connection between two vertices. An edge between vertices v_i and v_j (where i and j are indices identifying the vertices) is written as the pair (v_i, v_j).

A Simple Graph: G = (V, E)



V = {1,2,3,4} E = {(1,2),(1,3),(2,3),(3,4)}

7.4 Graph Theory: Foundations III

Reading the example graph. In the figure above, we have four vertices $V = \{1, 2, 3, 4\}$ and four edges $E = \{(1, 2), (1, 3), (2, 3), (3, 4)\}$.

- Vertex 1 has a connection to vertices 2 and 3, but *not* to vertex 4. This means, for example, that if junctions 1 and 4 in a road network were represented by these vertices, you could not travel directly between them — you would have to pass through vertex 3 first.
- Vertex 3 is the most "central" vertex here: it connects to all other vertices. In the road-network analogy, this junction is the busiest, as all traffic must pass through it to get between the left cluster ($\{1, 2\}$) and vertex 4.
- Vertices 1 and 2 are connected to each other. In graph terminology this means there is a *direct* path between them.

Key Insight

A graph is an abstraction. The same mathematical structure can represent road networks, social networks, computer networks, molecular structures, or any collection of entities with pairwise relationships. The algorithms we develop for one domain (e.g. finding shortest paths in a road network) transfer directly to other domains.

7.4 Weighted Graphs I

In a plain (unweighted) graph, an edge simply records that two vertices are connected. In a **weighted graph**, each edge also carries a numerical **weight** — a number that quantifies the "cost" or "strength" of the connection.

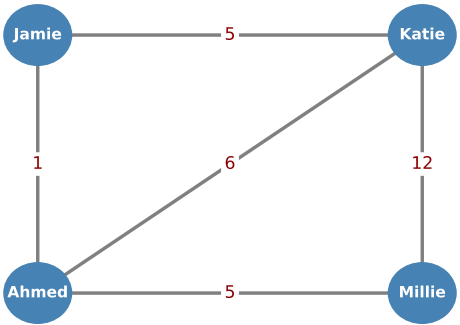
Definition: Weighted Graph

A weighted graph $G = (V, E, W)$ extends the basic definition with

- W (capital W): a **weight function** $W : E \to \mathbb{R}$ that assigns a real number to each edge. In road networks this weight is typically the *distance* in metres or kilometres, or the *travel time* in seconds. In social networks it might be the *level of interaction* between two users.

7.4 Weighted Graphs II

Weighted Graph: Social Network



Edge weight = level of activity between users

7.4 Weighted Graphs III

Reading the social-network example. The four users Jamie, Katie, Ahmed, and Millie form a weighted graph in which the edge weight represents the frequency of interaction. Jamie and Katie have weight 5 (moderate activity). Katie and Millie have weight 12 (the highest in the network — they interact most frequently). Jamie and Ahmed have weight 1 (rare interaction). Notice that *Millie and Jamie are not connected by an edge at all*, meaning no recorded interaction between them.

7.4 Weighted Graphs IV

Weighted graphs for delivery optimisation. In a road network weighted graph:

- Each **node** represents a road junction (intersection).
- Each **edge** represents a road segment connecting two junctions.
- The **weight** of an edge is typically the road distance (in metres) or the estimated travel time.
- Additional edge attributes might include the road name, classification (motorway, A-road, footpath), and speed limit.
- Node attributes might include whether the junction has traffic lights, or whether it is a roundabout.

When a routing algorithm (such as Dijkstra's algorithm) searches for the *shortest path* between two locations, it traverses this weighted graph and finds the sequence of edges whose total weight (total distance or travel time) is minimised.

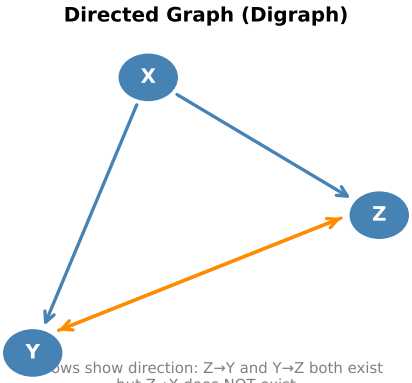
7.4 Directed Graphs I

In an **undirected** graph, an edge (v_i, v_j) implies that you can travel from v_i to v_j *and* from v_j to v_i — the connection is symmetric. Real road networks, however, have **one-way streets**: you may drive north along a street but *not* south. To model this, we use a **directed graph** (also called a *digraph*).

Definition: Directed Graph

In a directed graph, each edge (v_x, v_y) has a specific **direction**: it goes *from* v_x *to* v_y, but *not* necessarily the reverse. The existence of edge (v_x, v_y) does not imply the existence of edge (v_y, v_x). When both directions exist, two separate directed edges are needed.

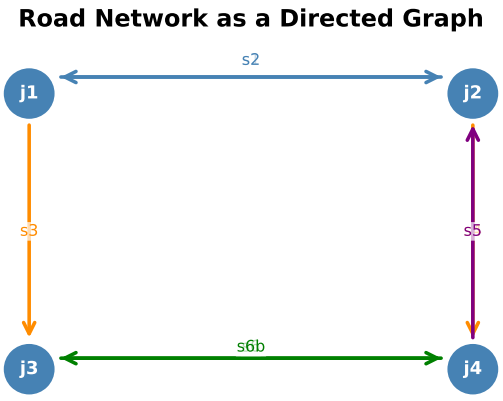
7.4 Directed Graphs II



7.4 Directed Graphs III

Reading the directed graph. In the diagram, vertex X connects to both Y and Z (arrows pointing outward from X). Between Y and Z there are edges in *both* directions (Y→Z and Z→Y), shown as two separate arrows. Crucially, there is *no* edge from Z or Y back to X, and no edge from X directly to Y via Z. A vehicle at Z cannot reach X in one hop.

Two edges for a two-way street. For a bidirectional road segment between junctions j_1 and j_2, the graph model requires *two* directed edges: $j_1 \to j_2$ and $j_2 \to j_1$. Both can carry the same weight (road distance), but they could carry different weights if travel is faster in one direction (e.g. due to traffic light phasing).



j1–j4 = junctions (nodes); s1–s6 = road segments (edges)
Two edges between j1↔j2 and j3↔j4 for bidirectional roads

7.4 Directed Graphs V

The road network graph with four junctions j_1–j_4 connected by directed road segments s_1–s_6. Two directed edges between $j_1 \leftrightarrow j_2$ model a two-way road; a single directed edge models a one-way street.

Edge Attributes in a Road Network Graph

When building a road network graph from vector data (e.g. OSM), the edges typically carry the following attributes:

- **Street name**: e.g. "George Street"
- **Road number**: e.g. "A1"
- **Road classification**: Motorway, Trunk, Primary, Secondary, Residential, Footpath, etc.
- **Speed limit**: in km/h or mph
- **Length**: in metres (computed from the GPS coordinates of the edge endpoints using the Haversine formula)

Junction *nodes* may carry:

- **Traffic lights**: true/false
- **Roundabout**: true/false

7.4 Directed Graphs VII

Key Insight

By storing a road network as a directed, weighted graph with rich edge attributes, we separate the *data model* (the graph) from the *algorithm* (the routing engine). This separation means we can swap in a different routing algorithm without changing the underlying data structure.

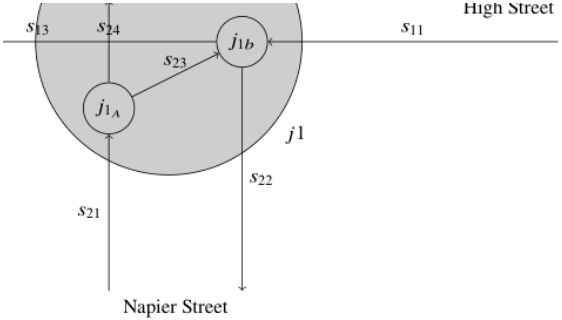
7.4 Complex Junctions and Turn Restrictions I

Simple junctions can be modelled with a single node. However, many real-world junctions have **restricted turns**: at a T-junction or crossroads, certain turn combinations may be prohibited — for example, "no right turn from Napier Street onto High Street". A single node cannot represent this, because the node model does not distinguish between entering the junction from different directions.

Modelling Complex Junctions with Multiple Nodes

A complex junction j_1 can be *expanded* into a small sub-graph of internal nodes and edges, where:

- Each internal node represents a particular *approach direction* into the junction (e.g. j_{1a} = arriving via Napier Street, j_{1b} = arriving via High Street eastbound, j_{1c} = arriving via High Street westbound).
- Internal edges represent *permitted* movements through the junction.
- A prohibited turn is simply absent — there is no internal edge for that movement.



ion, with restricted turns represented using a graph

then proceed to j_{1b} for a left turn onto High Street or j_{1c} for a
Street. The arrangement does not allow traffic proceeding along
a right turn into Napier Street. It also ensures that traffic on High
m a "U turn"—e.g. from s_{11} to s_{12}. By using multiple nodes and
tion, complex junctions can be modelled.
ber of ways in which we can represent graphs as a data structure.

7.4 Graph Data Structures I

We now know *conceptually* what a graph is. The question is how to represent it efficiently in computer memory. There are two principal choices: the **object-oriented representation** and the **adjacency matrix**.

Option 1: Object-Oriented (Node and Edge Classes)

The most natural approach from an object-oriented programming perspective is to define classes `Node`, `Edge`, and `Graph`:

- A `Node` object stores its identifier, coordinates, and a list of incident edges.
- An `Edge` object stores its weight, its source `Node`, and its destination `Node`.
- A `Graph` object holds a list of all nodes and a list of all edges.

This design is clean and intuitive. However, for a major city's road network (potentially tens of thousands of nodes and edges), the overhead of managing tens of thousands of individual Python objects — and navigating between them via object references — can make operations slow.

Option 2: Adjacency Matrix

An alternative representation uses a 2D matrix (a 2D array or list-of-lists in Python) of size $n \times n$, where n is the number of nodes in the graph.

- The cell at row i, column j of the matrix stores 1 (or the edge weight) if there is an edge from node v_i to node v_j.
- If there is no direct connection between v_i and v_j, the cell stores 0 (or -1 to signal "no direct link").

Advantages:

- Checking whether two nodes are connected is an $O(1)$ operation (a single array lookup).
- Finding all neighbours of a node is an $O(n)$ scan of one row.
- Numerically efficient — a large array is far cheaper in memory and access time than a large collection of Python objects.

Disadvantage: For a graph with n nodes, the matrix uses n^2 cells. A network with 10,000 nodes requires a 100,000,000-cell matrix, which consumes a great deal of memory even if most roads do not directly connect to each other (i.e. the matrix is *sparse*).

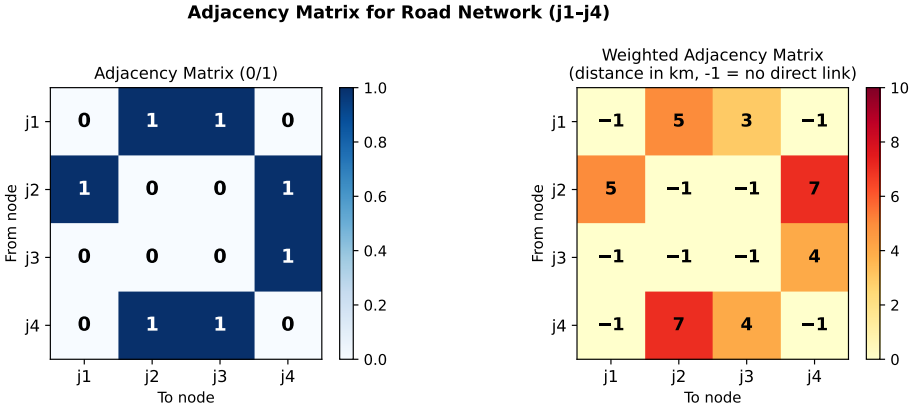
Worked Example: Adjacency Matrix for j_1–j_4

Consider the directed road network with junctions j_1, j_2, j_3, j_4 (Fig. 7.9 from the book). The directed edges are: $j_1 \to j_2$, $j_2 \to j_1$ (bidirectional top road), $j_1 \to j_3$ (one-way left road), $j_2 \to j_4$ (one-way right road), $j_4 \to j_2$ (one-way returning), $j_3 \leftrightarrow j_4$ (bidirectional bottom road).

	j1	**j2**	**j3**	**j4**
j1	0	1	1	0
j2	1	0	0	1
j3	0	0	0	1
j4	0	1	1	0

Row i tells you where node j_i can *go to*. Column j tells you which nodes can *reach* j_j. For example, row j3 has only one "1" (in column j4), confirming that the only direct exit from junction j_3 is to j_4. Cell (j1, j4) is 0, confirming no direct road from j_1 to j_4.

7.4 Graph Data Structures IV



7.4 Graph Data Structures V

Weighted adjacency matrix. A useful modification is to replace the binary 0/1 entries with actual edge weights. For roads this would be the distance in metres. Cells where no direct connection exists are set to -1 (to distinguish "no connection" from a zero-length edge).

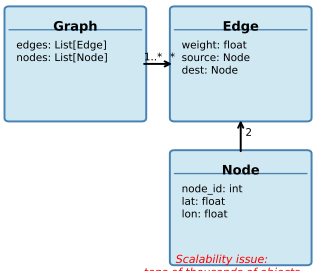
Using the Weighted Adjacency Matrix

- **Checking connectivity**: cell $(i, j) > 0$ means there is a direct road from junction i to j.
- **Getting the distance**: cell (i, j) gives the road length directly.
- **Finding neighbours**: scan row i and collect all column indices j where the cell is > 0.
- In many routing algorithms (e.g. Dijkstra's), the adjacency matrix is the primary lookup table: at each step, the algorithm reads $A[i][j]$ to get the cost of moving from junction i to j.

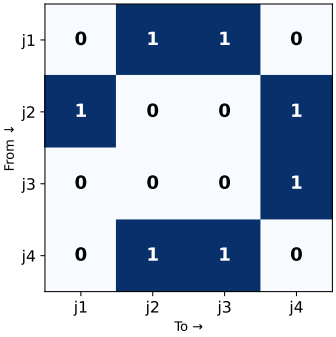
7.4 Graph Data Structures VI

Graph Data Structure Representations

Object-Oriented Design



Adjacency Matrix (compact, fast)



O(1) lookup: A[i][j] tells us directly if j1 connects to j2

Python: Building a Graph with an Adjacency Matrix

Listing 4: Graph class using an adjacency matrix

```
import numpy as np

class Graph:
    """
    A directed, weighted graph represented by an adjacency matrix.
    Nodes are labelled 0 to n-1 internally.
    """
    def __init__(self, n: int):
        self.n = n
        # -1 means no direct connection; 0 or positive means there is one
        self.adj = -1 * np.ones((n, n), dtype=float)
        np.fill_diagonal(self.adj, 0)    # Distance from a node to itself = 0

    def add_edge(self, src: int, dst: int, weight: float):
        """Add a directed edge from src to dst with the given weight."""
        self.adj[src][dst] = weight

    def get_weight(self, src: int, dst: int) -> float:
        """Return edge weight, or -1 if no direct connection."""
        return self.adj[src][dst]

    def neighbours(self, node: int) -> list:
        """Return list of (neighbour_index, edge_weight) pairs."""
        return [(j, self.adj[node][j])
                for j in range(self.n)
```

Python: Building a Road Network with OSMnx

In practice, we rarely build graph objects by hand. The `osmnx` library downloads road network data from OpenStreetMap and constructs a `networkx` directed graph automatically. Each node corresponds to a road junction and carries latitude/longitude attributes; each edge corresponds to a road segment and carries distance, speed limit, and other metadata.

Listing 5: Building and querying an OSM road network graph

```
import osmnx as ox
import networkx as nx

# Download the driveable road network for central Edinburgh
G = ox.graph_from_place("Edinburgh City Centre, UK",
                        network_type="drive")

# G is a networkx MultiDiGraph (directed, multi-edge)
print(f"Nodes: {G.number_of_nodes()}")
print(f"Edges: {G.number_of_edges()}")

# Get node attributes (latitude and longitude)
node_id = list(G.nodes)[0]
attrs = G.nodes[node_id]
print(f"Node {node_id}: lat={attrs['y']:.5f}, lon={attrs['x']:.5f}")

# Get edge attributes
u, v, key, data = list(G.edges(keys=True, data=True))[0]
print(f"Edge {u}->{v}: length={data.get('length', '?')} m, "
```

7.4 Road Network as a Graph: Edinburgh Example I

The following figure shows a satellite image of part of Edinburgh alongside the graph representation generated from OpenStreetMap data for the same area. Each blue dot is a node (road junction); each line is an edge (road segment).

7.4 Road Network as a Graph: Edinburgh Example II

(a) A map showing the road network in the centre of Edinburgh.

7.5 Conclusions I

This chapter has described the data types and structures that are commonly used in geographical information systems. These form a necessary part of our problem models when we come to optimise delivery routes.

7.5 Conclusions II

Summary of Key Concepts

1. **History of mapping**: The desire to create accurate maps is ancient, but the digital revolution has made planetary-scale, on-demand geospatial data available to everyone. OpenStreetMap is a freely available, volunteer-built global map used widely in delivery and logistics applications.
2. **Raster vs. vector**: Map data may be stored as raster pixel images or as vector geometric data. A rendering algorithm converts vector data into the raster images that screens display. Vector data is preferred for GIS applications because it is compact, scalable, and can carry rich metadata.
3. **Coordinate systems**: The latitude/longitude system (WGS84) provides a global framework for specifying locations. The Haversine formula gives the great-circle (straight-line surface) distance between any two coordinate points on the Earth.
4. **Graph theory**: A directed, weighted graph $G = (V, E)$ is the natural data structure for representing a road network. Junctions become nodes, road segments become directed edges, and distances or travel times become edge weights. The adjacency matrix provides a computationally efficient implementation.

7.5 Conclusions III

Looking Ahead

With the road network stored as a graph, we are ready to apply **routing algorithms** — methods for finding efficient paths through the graph. Chapter 8 will introduce the most important such algorithms, including Dijkstra's shortest-path algorithm, and explain how they are used within delivery optimisation frameworks.

7.5 Conclusions IV

Practical Python Ecosystem for GeoSpatial Work

The following Python libraries together provide a complete pipeline from raw geospatial data to an optimisable graph:

- **osmnx**: downloads road networks from OpenStreetMap and returns `networkx` graph objects.
- **networkx**: general-purpose graph library; includes Dijkstra's algorithm, betweenness centrality, and many other graph algorithms.
- **shapely**: geometric operations on vector shapes (intersection, union, buffering).
- **geopandas**: extends pandas DataFrames with a geometry column; supports reading/writing all major vector formats (GeoJSON, Shapefile, etc.).
- **folium** / **leaflet.js**: interactive map visualisation in a browser.
- **matplotlib**: static map plots, graph diagrams.

Python: End-to-End Pipeline Summary

Listing 6: Complete pipeline: OSM data to shortest-path result

```
import osmnx as ox
import networkx as nx
import math

def haversine(lat1, lon1, lat2, lon2):
    R = 6371.0
    phi1, phi2 = math.radians(lat1), math.radians(lat2)
    dphi = math.radians(lat2 - lat1)
    dlam = math.radians(lon2 - lon1)
    a = math.sin(dphi/2)**2 + math.cos(phi1)*math.cos(phi2)*math.sin(dlam/2)**2
    return R * 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))

# Step 1: Download road network from OSM
G = ox.graph_from_place("Edinburgh, UK", network_type="drive")
print(f"Network: {G.number_of_nodes()} nodes, {G.number_of_edges()} edges")

# Step 2: Find the nearest graph nodes to two address coordinates
# (lat, lon) of two delivery locations
delivery_A = (55.9533, -3.1883)    # Edinburgh city centre
delivery_B = (55.9441, -3.2024)    # Tollcross area

node_A = ox.nearest_nodes(G, X=delivery_A[1], Y=delivery_A[0])
node_B = ox.nearest_nodes(G, X=delivery_B[1], Y=delivery_B[0])

# Step 3: Compute shortest path by road distance
```

Chapter 7 Summary Card

Concepts

- Raster: pixel grid, resolution-limited
- Vector: geometry + metadata, scalable
- WGS84: global lat/lon coordinate standard
- Haversine: great-circle distance formula
- Graph $G = (V, E)$: nodes + edges
- Weighted graph: edges carry distances/times
- Directed graph: models one-way streets
- Adjacency matrix: $O(1)$ connectivity lookup

Key Formulas

Haversine:

$$a = \sin^2 \frac{\Delta\phi}{2} + \cos\phi_1 \cos\phi_2 \sin^2 \frac{\Delta\lambda}{2}$$

$$d = 2R \operatorname{atan2}\left(\sqrt{a}, \sqrt{1-a}\right)$$

Graph definition:

$$G = (V,\ E)$$

V = vertices (junctions), E = edges (roads)

Python Libraries

`osmnx`, `networkx`, `geopandas`, `shapely`, `folium`